Software Development Life Cycle (SDLC)

RCS 305 ? Software Engineering | Riara University Revision Resource

1. Introduction & Overview

The Software Development Life Cycle (SDLC) is a structured framework that guides the conception, specification, development, verification, deployment, and operational maintenance of reliable software systems. By enforcing defined milestones and quality gates between phases, SDLC disciplines prevent runaway defect costs and align engineering outputs with institutional and stakeholder requirements.

2. Core Architectural Phases

A standard software engineering lifecycle comprises five core stages:

? Requirement Analysis: Eliciting user needs, evaluating operational constraints, and drafting unambiguous Functional & Non-Functional Specifications.

? Architectural Design: Structuring multi-tier architectures, data schemas, module cohesion, interface contracts, and component decoupling.

? Implementation: Coding clean, documented, and modular routines adhering to established language guidelines and peer review standards.

? Verification & Testing: Subjecting software to unit, integration, system, and user acceptance test suites to detect regressions before release.

? Evolution & Maintenance: Addressing post-deployment defects, adapting to updated operational runtime environments, and enhancing capabilities.

3. The Defect Amplification Model (Boehm’s Law)

Empirical studies demonstrate that defect remediation costs increase exponentially across successive SDLC phases. A requirement ambiguity resolved during initial analysis costs approximately 1x to fix. The same defect discovered during coding costs 5x, during integration testing costs 10x, and post-deployment in production exceeds 30x to 100x. Systematic upstream verification is therefore essential.

4. Multi-Tier Operational Environments

Enterprise deployment mandates strict physical or logical separation between operational environments:

? Development (Dev): Dynamic, unstable staging environment for rapid prototyping and unit testing.

? Quality Assurance (QA): Isolated replica of production infrastructure executing automated regression suites, load testing, and UAT sign-offs.

? Production (Prod): Live, monitored environment delivering services to authenticated end users under rigorous change control.

MaxLabs Academic Revision Document -- Copyright Riara University Design Thinking Prototype 2026